in xalancbmk on machine 2. There was also a 0.5 ∼ 1.5% consistent slowdown in gcc, mcf, and xalancbmk.

For the LLVM Nightly Tests, the worst slowdown was 4.3%, and the best speedup was 4.0%. On average, we observed a 0.3% slowdown. One benchmark, Oscar, had a huge speedup (25%). This was because our fixes prevented loop vectorization from working in this case, which happened to make the benchmark run faster. This benchmark is omitted in the numbers mentioned before.

In the LLVM Nightly Tests, only 25% (464/1853) of object files were different than when compiled with the baseline compiler, and only 27% (80/293)of benchmarks had any object file changed.

Across all of our benchmarks, the baseline version of LLVM's GVN pass performs about 24,700 replacements. About 28% of these correspond to propagation of pointer equalities, with the remaining being propagation of integer equalities. Of the pointer replacements, 44% correspond to replacing a pointer with `NULL`, which is a sound transformation under our semantics (even though it was disabled in this prototype).

Performance after adding optimizations. Fig. 7b shows the change in performance of SPEC CPU 2017 for our second prototype ("sound and fast") relative to baseline LLVM. The three changes that contributed the most to recovering lost performance were as follows. Adding **psub** changed the average slowdown on SPEC from −0.2% to −0.1%. The main winners due to this optimization were mcf and xalancbmk, having a 1.7% and 1.1% speedup, respectively.

We found "jpeg-6.a" in LLVM Nightly Tests showed 2.2% slowdown. It was because removing transformations from load/store of pointers into load/store of pointer-sized integers blocked SLP vectorization of heterogeneous types. We can support this by allowing type punning between a physical pointer and integer. Formally, we can define load of integer i as pointer to yield $\mathsf{Phy}(i, 0, \emptyset, \textbf{None})$ rather than **poison**, and load of $\mathsf{Phy}(i, 0, \emptyset, \textbf{None})$ as integer to return i. With this update, it is valid to canonicalize load/store of pointer-sized integers into load/store of pointers. After the canonicalization, SLP vectorization can fire because load/stores now have homogeneous types. We implemented this canonicalization pass before SLP vectorization and checked that the slowdown disappeared. It did not, however, improve the performance of the SPEC benchmarks nor the average slowdown of the LLVM Nightly Tests. This canonicalization can be a good workaround for reenabling vectorization, but in the future we would like to pursue more generalized solution for this, as discussed in Section 8.

Finally, reactivating GVN for safe cases removed the slowdown in perlbench and cactuBSSN. With the Nightly Tests, the performance regressed slightly, increasing the average slowdown to 0.07%.

This experiment shows that it is possible to make LLVM adhere to our memory model without significant performance regression. In fact, some benchmarks get a small performance improvement.

6.3 Number of Instructions

We investigated how the number of pointer cast instructions (**inttoptr** / **ptrtoint**) and **psub** instructions evolved with our modifications. Table 1 shows the total number of instructions over all the benchmarks, when compiled without optimizations (`-O0`) and with optimizations (`-O3`).

In the first column, we have the numbers for the baseline (vanilla LLVM). In the second column, we can observe the number of casts does not increase significantly after disabling the (incorrect) optimization that removes the round-trip cast of `(int*)(int)p` to `p`, for a pointer `p`.

The third column shows the result for our first prototype, where all incorrect optimizations were removed. The number of **inttoptr** instructions is reduced because we disabled the introduction of type punning by the optimizer. The increase in the number of **ptrtoint** casts is the result of removing unsound optimizations like the one that rewrites `(int)p == (int)q` into `p == q`.